Parsing Static Web Pages

Week 6 · Documents module · Textbook Chapter 6

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 21, 23 & 25, 2026

This week at a glance

We turn HTML — the language of web pages — into data you can analyze.

Monday | Concepts & notebook intro

- From HTML to a DataFrame; three scraping strategies

Wednesday | Notebook lab

- Work through and share the Chapter 6 notebook: box office, Wikipedia, and the Oscars

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 6

Companion notebook

`ch-06-static-pages.ipynb`

Bring a laptop

Wednesday is hands-on — we debug live.

1. Monday · Concepts

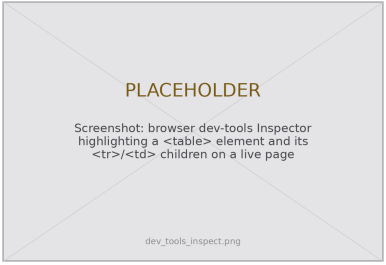
From HTML to data

The goal of scraping: convert one structured format — HTML — into another suited to analysis (a pandas DataFrame, CSV, or JSON).

HTML and XML are **siblings** (both descend from SGML), so your `BeautifulSoup` skills transfer directly.

But browsers *tolerate* malformed markup — unclosed tags, missing quotes, bad nesting — so real HTML is far messier than clean XML.

The core skill this week: **cutting through the noise** of navigation bars, ads, tracking scripts, and nested `<div>`s to reach the data.

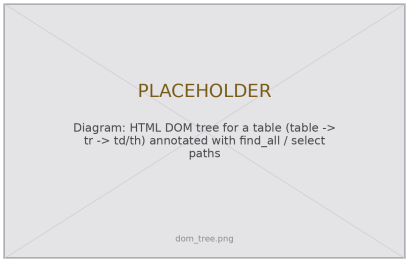


Inspect first, code second.

Textbook Ch. 6, "From HTML to Data."

Three strategies, increasingly general

1. **Manual table parsing** — walk `<table>` → `<tr>` → `<td>` by hand. Most control, most code.
2. `pandas.read_html()` — one call grabs every table on a page. Least code, least control.
3. **Custom parser** — navigate the DOM tree for data that isn't in a table (`<div>`s + CSS classes).



Choose the *simplest* strategy that fits the page — and be ready to fall back to a more general one.

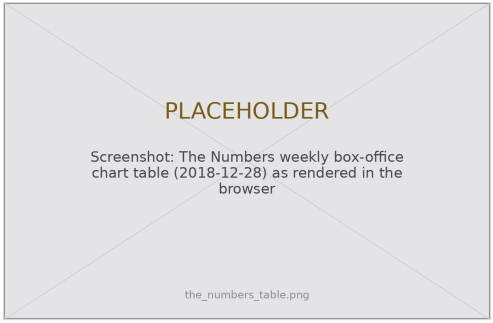
Textbook Ch. 6.

Inspect before you parse

Every scrape starts in the browser's developer tools, not in Python.

- Right-click the data → **Inspect**
- Find the `<table>`, or the repeating `<div>` container
- Note the tags, `class` names, and `id`s that anchor your target
- "Copy selector" gives you a CSS path to paste into `select()`

The page is a **hypothesis**: inspect, guess an anchor, verify the count, refine.



The Numbers weekly box-office chart — our first target.

Textbook Ch. 6, "Retrieving and Inspecting."

What the notebook covers

This week's companion notebook builds one skill at a time:

- **Strategy 1 —** box-office revenue from The Numbers, cell by cell
- **Strategy 2 —** highest-grossing films from Wikipedia with `read_html()` + cleanup
- **Strategy 3 —** Oscar nominees from nested `<div>`s, then Colorado legislators
- **Reuse —** abstract a working parser into a function; rate-limit; handle errors

Connecting to Ch. 2 (Ethics)

Before pointing a loop at any site, check its `robots.txt` and Terms of Service, and reuse `responsible_get()` from the ethics chapter. Politeness is not optional.

2. Wednesday · Notebook Lab

Strategy 1 — manual table parsing

Retrieve, then find the right table (pages hide data tables among navigation tables):

```
import requests
from bs4 import BeautifulSoup

url = "https://www.the-numbers.com/box-office-chart/weekly/2018/12/28"
headers = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
soup = BeautifulSoup(requests.get(url, headers=headers).text, "html.parser")

tables = soup.find_all("table")
for i, table in enumerate(tables): # inspect each candidate
    print(i, len(table.find_all("tr")), "rows")
```

Textbook Ch. 6, Strategy 1.

The one-line bug that shifts every column

Tempting shortcut:

```
# DON'T: silently drops empty cells
row.text.split("\n")
```

Correct — walk cell by cell so **empty cells stay aligned**:

```
cells = row.find_all(["td", "th"])
values = [c.get_text(strip=True)
          for c in cells]
```

A blank header cell (the “previous rank” column) is easy to lose.

Split-on-newline drops it — and every value after shifts one column left, so **Movie** quietly fills with distributor names.

The extra line of code buys correctness.

Textbook Ch. 6, “Extracting Row Data.”

Strategy 2 — `pandas.read_html()`

```
import pandas as pd

url = "https://en.wikipedia.org/wiki/2018_in_film"
tables = pd.read_html(url) # every table on the page -> list of DataFrames
films = pd.read_html(url, header=0)[3] # pick the right one by index
```

Powerful, but rarely perfect. Expect to spend as long **cleaning** as extracting:

```
df = df[df.iloc[:, 0] != df.columns[0]] # drop repeated header rows
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
df["worldwide_gross"] = pd.to_numeric( # "$1,234" -> 1234
    df["worldwide_gross"].str.replace(r"[\$,]", "", regex=True), errors="coerce")
df["studio"] = df["studio"].ffill() # fill merged (colspan) cells
```

Textbook Ch. 6, Strategy 2. `errors="coerce"` keeps the pipeline alive on bad cells.

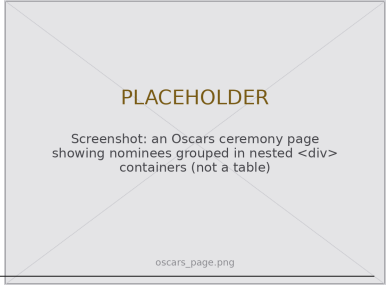
Strategy 3 — custom parser & CSS selectors

Non-tabular data (Oscar nominees) lives in nested `<div>`s. Find a reliable anchor, then search within it — `find()` + `find_all()`, or one CSS selector:

```
# navigate: up to a unique parent, then down to the repeating rows
area = soup.find("div", class_="field--name-field-award-categories")
categories = area.find_all("div", class_="view-grouping") if area else []

# equivalent, in one readable line (paste from "Copy selector"):
soup.select("div.field--name-field-award-categories div.view-grouping")
```

`select()` returns a list like `find_all()`; `select_one()` returns one like `find()`.



Textbook Ch. 6, Strategy 3.

Make it reusable — and defensive

Abstract the working parser into a function; keep the `sleep` in the *caller*, not the function:

```
for year in range(2020, 2025):
    df = scrape_oscar_year(year, headers) # returns empty DataFrame on failure
    all_years.append(df)
    time.sleep(1) # politeness lives in the loop
```

Wrap fragile parsing so one bad page doesn't kill the run:

```
try:
    response.raise_for_status()
except requests.RequestException as e:
    print(f"Request failed: {e}"); return None
# AttributeError fires when .find() returns None and you call .text on it
# -- the single most common scraping error.
```

Textbook Ch. 6, "Abstracting into Functions" & "Error Handling Patterns."

Lab: work these, then show-and-tell

Work through the notebook exercises in pairs:

1. Wikipedia table via `read_html` + a visualization
2. Custom parser for a non-tabular page (document your selectors)
3. Extend it to multiple pages (sleep, try/except, a function)
4. Non-English Wikipedia — what encoding cleanup is needed?
5. Cache-first: download once to `cache/`, parse offline
6. 5617: audit two sites' ToS & `robots.txt` vs. Fiesler et al. (2020)

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

Sites redesign without warning.
When a parser breaks, that's not a bug in your code —
it's the web working as designed.

Re-inspect, re-hypothesize, re-verify.

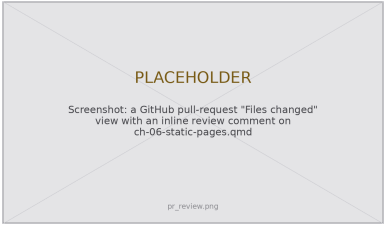
3. Friday · Textbook Revisions

Improving Chapter 6 together

Today we make the book better. Each of you opens a **pull request** against `ch-06-static-pages.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 6

High-value targets — pick one and scope it to a single PR:

- **Test the code against live sites.** The Numbers, Wikipedia, and Oscars pages drift. Run the notebook, report what broke, and fix the selectors (this is the chapter's own "A Note on Fragility").
- **Close a gap in "Common Issues to Debug."** Add a worked `\xa0` / non-breaking-space example, or a `colspan/rowspan` case `read_html` mishandles.
- **Add a figure.** A clean DOM-tree diagram, or an annotated Inspector screenshot, would help Strategy 1 and 3.
- **Strengthen an exercise.** Add expected output, a rubric, or a starter cell so exercises are self-checking.
- **Extend "Further Reading."** A data-journalism scrape (ProPublica, The Markup) tied to the "Public Interest" callout.

Targets drawn from the chapter's callouts, "Common Issues," and "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Archived web pages —** scraping the past with the Wayback Machine.

Key takeaways

1. Scraping is a pipeline: **retrieve** → **inspect** → **anchor** → **extract** → **DataFrame**.
2. Start with `read_html()` for tables; fall back to manual or custom parsing for control.
3. Navigate cell by cell — shortcuts silently misalign columns.
4. Abstract into functions, rate-limit in the loop, and code defensively.
5. Pages change: the durable skill is **inspect, hypothesize, verify** — not memorized class names.

Reading: Textbook Ch. 6. Related: Mitchell (2018); Vanden Broucke et al. (2018).